

AutoML 기능 설계 문서

PyCaret 기반 카드 결제 이상거래 탐지 기능

과제 : 웹 개발자도 쉽게 사용하는 AutoML — 분류 기능 설계도 작성

대상 기능 : 카드 결제 이상거래 탐지 (fds_radar_ai / card_model_automl)

작성자 : 혜원

작성일 : 2026-09-03

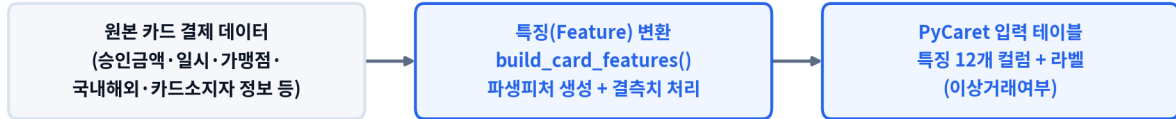
1. AutoML 기능 설계도

입력(원본 카드 결제 데이터) → 특징(Feature) 변환 → PyCaret 분류 파이프라인 → 결과 반환 → 백엔드/화면 연동까지의 전체 흐름은 아래와 같다. 본 기능은 이미지·텍스트가 아닌 정형(tabular) 거래 데이터를 입력으로 받아, 원본 필드를 파생 피처로 변환한 뒤 PyCaret 분류 파이프라인에 입력하는 구조다.

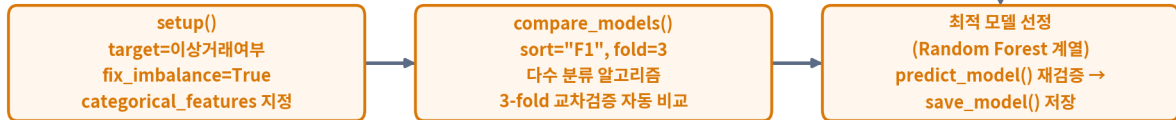
카드 결제 이상거래 탐지 — AutoML(PyCaret) 기능 설계도

입력(정형 거래 데이터) → 특징 변환 → PyCaret 분류 파이프라인 → 결과 반환 → 백엔드/화면 연동

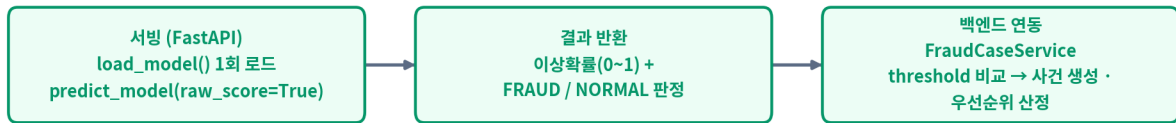
STEP 1 데이터 · 특징 설계



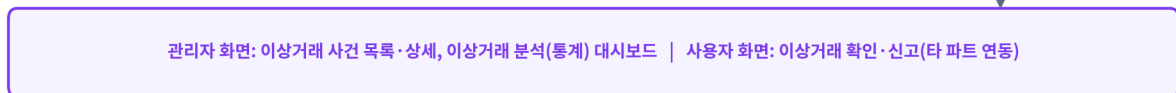
STEP 2 PyCaret 분류 파이프라인



STEP 3 서빙 · 결과 반환 · 백엔드 연동



STEP 4 화면 연계



[그림 1] 카드 결제 이상거래 탐지 — AutoML(PyCaret) 기능 설계도

2. 기능 정의

2.1 기능 개요

카드 결제 거래 1건이 발생했을 때, 해당 거래의 정형(tabular) 속성값을 바탕으로 이상거래 여부를 자동 분류하고 이상 확률(0~1)을 함께 산출하는 기능이다. PyCaret AutoML로 학습한 분류 모델을 FastAPI 서버에 배포하여, 실시간 거래 발생 시점에 예측을 수행한다.

2.2 입력 · 출력 정의

구분	내용
입력(대상)	카드 결제 거래 1건의 정형 데이터 (승인금액·승인일시·가맹점·국내/해외 여부·카드 소지자 정보 등)
분류 클래스(라벨)	이상거래여부 — 0(정상, NORMAL) / 1(이상, FRAUD) 의 이진분류
출력값	이상 확률(prediction_score, 0~1 연속값) + 이상거래 여부(FRAUD/NORMAL) 판정

2.3 프로젝트 내 연계 지점

- 카드 결제 API(POST /api/transactions/payment) 처리 후, 이상거래 탐지 API(POST /api/fraud-detections)가 호출되면 백엔드의 AutoMLFraudModelService가 FastAPI 서버의 /predict 엔드포인트를 호출한다.
- 산출된 이상 확률이 설정값(fraud.threshold, 현재 0.50)을 넘으면 관리자 조사 사건(FraudCase)이 자동 생성된다.
- 관리자 화면 — 이상거래 사건 목록/상세, 이상거래 분석(통계) 대시보드에 확률·우선순위·유형이 표시된다.
- 사용자 화면 — 본인 거래에 대한 이상거래 확인/신고 화면과 연동된다(해당 화면 자체는 타 파트 구현).

3. 데이터 · 특징 설계

3.1 데이터 수집

AI Hub에서 제공하는 금융 합성 데이터(가상 카드 거래 데이터)를 사용한다. 2023년 1분기 ~ 2024년 4분기까지 총 8개 분기 원본 CSV를 Training/Validation 폴더 기준으로 그대로 병합하여 사용하며, 임의(random) 분할이 아니라 제공된 파일 단위로 학습·검증 데이터가 이미 분리된 상태로 공급받는다라는 점이 특징이다.

구분	규모
검증(Validation) 데이터	총 105,942건
실제 이상거래 비율	3,687건 (3.48%) — 정상 거래 대비 이상거래가 매우 드문 불균형 데이터
학습(Train) 데이터	동일 방식으로 병합한 학습용 원본을 사용, 알고리즘 비교 단계에서는 클래스별 층화 20% 표본을 사용(속도 확보 목적)

3.2 특징(Feature) 변환 방식

이미지·텍스트가 아닌 정형 데이터이므로, 임베딩·TF-IDF 대신 도메인 지식 기반 파생 피처 생성 + 결측치 처리로 원본을 PyCaret 입력용 특징으로 변환한다 (build_card_features() 함수).

- 시간 특징: 승인일자 → day_of_week(요일), is_weekend(주말 여부) 파생
- 거래 맥락 특징: is_overseas(해외 결제 여부), is_new_merchant(신규 가맹점 여부), is_online(온라인 결제 여부)
- velocity 특징: 카드별 누적 거래 순번(card_txn_seq) — 카드 단위로 정렬 후 groupby-cumcount로 계산
- 결측치 처리: 연령은 중앙값(median)으로 대체, 성별(남녀구분코드)은 "UNKNOWN"으로 대체

3.3 PyCaret 최종 입력 데이터 형태 (특징 12개 컬럼 + 라벨 1개)

컬럼명	설명	구분
통합승인금액	카드 결제 승인 금액	수치형
승인시간대	결제가 발생한 시간대(0~23)	수치형
day_of_week	요일(0=월 ~ 6=일) — 승인일자에서 파생	수치형
is_weekend	주말 여부(0/1) — day_of_week에서 파생	이진
is_overseas	해외 결제 여부	이진
is_new_merchant	신규 가맹점 여부	이진
is_online	온라인 결제 여부	이진
연령	카드 소지자 연령 (결측 시 중앙값 대체)	수치형

컬럼명	설명	구분
남녀구분코드	성별 (결측 시 UNKNOWN 대체)	범주형
가맹점승인업종코드	가맹점 업종 코드	범주형
일시불할부구분코드	일시불 / 할부 구분	범주형
card_txn_seq	해당 카드의 누적 거래 순번 (velocity)	수치형
이상거래여부 (라벨)	0 = 정상, 1 = 이상 (예측 대상)	이진(타겟)

4. 모델링 · 예측 설계

4.1 PyCaret 분류 파이프라인 단계

단계	내용
① setup()	target="이상거래여부", session_id=42, fix_imbalance=True(오버샘플링으로 불균형 보정), categorical_features=[남녀구분코드, 가맹점승인업종코드, 일시불할부구분코드] 지정. train 데이터는 클래스별 총화 20% 표본, test_data는 검증 파일 전체를 그대로 홀드아웃으로 지정.
② compare_models()	sort="F1", fold=3 — PyCaret이 내장한 다수의 분류 알고리즘(Random Forest, LightGBM, Logistic Regression, Decision Tree 등)을 3-fold 교차검증으로 자동 비교.
③ 모델 선정	F1-score 기준 최상위 모델(n_select=1) 채택 → predict_model()로 검증 데이터 재확인 → 최종 Random Forest 계열 모델 선정.
④ save_model() / load_model()	선정된 모델(전처리 파이프라인 포함)을 .pkl로 직렬화 저장. FastAPI 서버 기동 시 load_model()로 1회만 메모리에 로드.
⑤ predict_model()	실시간 거래 1건에 대해 predict_model(data, raw_score=True) 호출 → prediction_score_1(이상 확률) 반환.

4.2 최적 모델 선정 기준

실제 이상거래 비율이 3.48%에 불과한 불균형 데이터이므로, Accuracy만으로는 "전부 정상"이라고만 예측해도 96%대의 수치가 나오는 착시가 발생한다. 따라서 Precision(이상으로 예측한 것 중 실제 정답 비율)과 Recall(실제 이상거래 중 잡아낸 비율)을 함께 반영하는 F1-score를 최적 모델 선정 기준으로 사용했다 (compare_models(sort="F1")).

4.3 최종 검증 성능 (Validation 105,942건 기준)

평가지표	값	의미
Accuracy	98.57%	전체 판단 정확도 (불균형 데이터 특성상 참고용 지표)
Precision	78.67%	이상거래로 예측한 것 중 실제 정답 비율
Recall	80.91%	실제 이상거래 중 모델이 잡아낸 비율
F1-score	79.78%	Precision·Recall을 종합한 최종 채택 기준 지표

4.4 예측 결과의 화면 표시 방식

- 이상 확률을 %로 환산해 표시 (예: 87%)
- 확률 구간에 따라 우선순위 자동 분류 — 0.90 이상 HIGH, 0.80~0.90 MEDIUM, 나머지(threshold~0.80) LOW
- 이상거래 유형은 SI가 직접 분류하지 않고, 백엔드가 해외 결제·신규 거래 여부 등을 규칙 기반으로 추론해 함께 표시
- 관리자 화면(이상거래 사건 목록·상세, 이상거래 분석 통계)에 확률·우선순위·유형·상태를 표/카드 형태로 표시

5. 설계 타당성 및 개선 방향

5.1 AutoML(PyCaret) 채택이 이 기능에 적합한 이유

- 금액·시간·가맹점·카드 소지자 특성 등 다양한 정형 피처 간 비선형 조합을 함께 봐야 하는 이진분류 문제로, 데이터 특성에 따라 알고리즘별 성능 편차가 클 수 있음.
- PyCaret은 전처리(불균형 보정·결측치·인코딩) 표준화, 다수 알고리즘 자동 비교, 배포용 직렬화까지 하나의 파이프라인으로 처리해 4인·2개월의 짧은 개발 기간 안에 안정적인 성능의 모델을 빠르게 확보하는 데 적합했음.
- `compare_models()`로 매번 여러 알고리즘을 직접 구현·튜닝하지 않고도 F1 기준 최적 모델을 자동 탐색할 수 있어, 모델링 전담 인력 없이도 실무 수준의 분류기를 구축할 수 있었음.

5.2 현재 설계의 한계

- 학습-서빙 불일치 (velocity): 학습 시 `card_txn_seq`는 전체 이력 기준 누적 순번(`cumcount`)으로 계산되지만, 실시간 서빙 코드는 이 값을 항상 고정값 1로 전송한다(TODO 주석 존재) — velocity 신호가 실제 서비스에는 반영되지 않고 있음.
- 학습-서빙 불일치 (연령·성별): 서빙 시점에 연령은 항상 30, 성별은 항상 UNKNOWN으로 고정 전송되어, 학습 때 있던 신호가 실서비스에서는 무력화된 상태.
- 임계값 이원화: AI 서버 자체 판정 기준(0.5, 코드에 고정)과 백엔드 사건화 기준(`fraud.threshold`, 설정값)이 분리돼 있어 혼동 여지가 있음. 이번에 검증 데이터 정밀 분석(`precision_recall_curve`)으로 F1 최적점(0.47)에 가장 가까운 값으로 0.70 → 0.50으로 재조정함.
- 데이터 불균형: 이상거래 비율 3.48%가 여전히 Precision(78.67%)을 제약하는 요인으로 작용함.

5.3 정확도 향상을 위해 고려할 점

- velocity·연령·성별 피처를 실시간으로 정확히 계산해 서빙에 반영하도록 백엔드-AI 서버 연동 로직 보완
- 현재 `threshold(0.50)`는 F1 최적점(0.47)에 가장 가까운 값으로 재조정된 것으로, 운영 목적(오탐을 더 줄일지·미탐을 더 줄일지)에 따라 주기적으로 재산정할 필요가 있음
- 오탐(False Positive)·미탐(False Negative) 사례를 지속적으로 수집해 재학습 데이터로 반영하는 피드백 루프 구축
- 이상거래 데이터(3.48%) 자체를 추가 확보하거나, 비용민감학습(`cost-sensitive learning`)·양상불 기법 등으로 소수 클래스 탐지 성능을 추가 개선